Enhancement of Bubble and Insertion Sort Algorithm Using Block Partitioning

Tithi Paul[1] †
[1]Department of Computer Science and Engineering, University of Barishal, Barishal-8200, Bangladesh
†tithi.cse3.bu@gmail.com

***Abstract*— A list of components can be arranged in a certain order using a sorting algorithm, which is a fundamental concept in computer science. The temporal complexity of the two fundamental and widely used sorting algorithms, Bubble sort and Insertion sort is $\mathcal{O}(N^2)$, where N is the total number of items. When it comes to sorting a specific amount of items, it is superior. However, by adding more parts to its quadratic complexity, it loses efficiency. Because of this, it is less frequently employed in computer science's practical and real-world applications, despite being widely utilized as a subroutine in other areas. Numerous extension techniques for the insertion sort and bubble sort algorithms have been put out in the literature, but none of them tries to combine the two to create a combination algorithm like ours. The bubble and insertion sort method was modified in this study, and its computational complexity was estimated to be $\mathcal{O}(N\sqrt{N})$. The technique begins by dividing the input array into a few pieces, sorting each of the blocks using a modified bubble sort, and then merging all of the blocks together using a modified insertion sort. The suggested bubble and insertion sort outperform traditional bubble and insertion sorting as well as all other sorting algorithms with a computational complexity of $\mathcal{O}(N^2)$.**

***Index Terms*—Sorting, Algorithm, Bubble Sort, Insertion Sort, Complexity Analysis**

I. Introduction

Since sorting is a fundamental algorithm, it may be found practically everywhere. Sorting is the action or procedure of arranging information or components in either ascending or descending order in accordance with predetermined criteria [1]. Sorting has long been considered one of the most fundamental and basic concepts in computer science. It is utilized in database systems as well as searching, networking, data transmission, pattern matching, and data analytics [2]. Strong sorting algorithms including merge sort, selection sort, bubble sort, insertion sort, and others may be found in the literature on computer science. However, insertion sort and bubble sort are two well-known sorting algorithms because they are simpler to develop and simulate than other sorting algorithms.

Many programmers find that using bubble or insertion sort is quite useful for a variety of programming tasks. The bubble and insertion sort, however, has a drawback. Both the bubble sort and the insertion sort have the worst-case time complexity of $\mathcal{O}(N^2)$[3]. When there are few elements, this is not a major issue, but as the number of elements grows, it becomes increasingly problematic. Although there are a few algorithms of $\mathcal{O}(Nlog(N))$ their implementation is not as straightforward as bubble and insertion sort [4]. In order to lower the time complexity, we suggested a novel approach that involved modifying the bubble and insertion sort algorithms. The overall time complexity of the proposed method is $\mathcal{O}(N\sqrt{N})$.

Using repeated effort, the sorting algorithm known as Bubble sort arranges the data. It evaluates every comparison between two neighboring elements. Change them if they aren't in the right sequence. It stops repeating its operation to indicate that the list is sorted when no swaps are needed. This method has a worst-case time complexity of $\mathcal{O}(N^2)$. Its effectiveness rapidly drops for lists with more numbers [12].

The sorted array is constructed by adding each element one at a time, using the Insertion sort method. By comparing the current value to all the items that came before its previous indexes, it places the value in the appropriate place. But regrettably, it has the same mathematical complexity as bubble sort $\mathcal{O}(N^2)$ in terms of time. Nevertheless, insertion sort has a number of benefits, including ease of understanding, minimal implementation, and performance for a number of tiny data sets [12][6]. Thankfully, this is no longer the case as sophisticated methods for data organization have been developed that do not need for the data to be maintained in a certain sequence [10]. For sorting the unordered lists, there are several well-known methods. The four that are most crucial are heap sort, bubble sort, insertion sort [16], and shell sort [11]. As mentioned in [12], sorting has been regarded as a fundamental algorithmic issue. The prevalence of sorting algorithms in beginning computer science courses serves as a gentle introduction to a number of fundamental algorithmic ideas [13][14]. That means all sorting Algorithms are problem specific, meaning they work well on some specific problem and do not work well for all the problems. On the other hand, an algorithm's relative efficacy and complexity are directly correlated [15].

Additionally, several efforts have already been done on the bubble and insertion sort that is already in use. In 2015, Ms. Jyoti Mundr and Mr. B. L. Pal suggested a novel approach for reducing the bubble sort algorithm's execution time. In order to produce a sorted array using the existing bubble sort, the first step compares the values for building the left and right sides of the input array [7]. Then, each value is sorted from leftmost to middle and from middle to rightmost. Moses mentioned the alternative bubble sort in 2009, known as the bidirectional bubble sort. It goes through the list and sorts from both end directions. The goal was to somewhat lower

979-8-3503-4602-2/22/$31.00 ©2022 IEEE

the average number of comparisons [8]. In a study by Jodoon, the selection sort algorithm was improved such that it starts sorting from both ends, discovers the largest and smallest data items in an array in a single iteration, and then inserts those at the appropriate spots. Repeat the process for the next two values, the second largest and the second smallest, and for the last iteration [9]. Although many of them have focused on improving the various sorting algorithms from various factors. The one that is being offered is different from all existing $\mathcal{O}(N^2)$ sorting algorithms in arithmetic [17].

The bubble and insertion sort algorithms don't have the same efficiency for a large number of items as our suggested technique has. Our primary contribution consists of two altered variants of the bubble and insertion sort known as Algorithm-1 and Algorithm-2. The first step is to partition the input array into the total square root number of blocks, and then use Algorithm-1 to sort each block separately. Several pieces of data are saved for each block in Algorithm-1 using a structure array called $Block$. The minimal unused value of each block is also included in the array along with the starting and terminating indexes. To create the final output array, Algorithm-2 is utilized. The current unused minimum number is located by searching and comparing all $\sqrt{N}$ blocks, after which it is added to the output array. The user's choice of how to organise the input array is up to them. If the user wishes to sort it in descending order, the minimum number should be swapped out for the maximum. Regarding sorting in ascending order, we will update the array with the next unused minimum number and continue the procedure until we have a complete output array that has been sorted.

The suggested approach is randomly tested on a variety of data sizes, from very tiny to very big. It has been noted that this strategy produces effective outcomes in terms of execution time. Observations were made by contrasting this strategy with the other algorithms listed in section IV.

The remainder of this essay will build on this foundation. Section II gives the suggested method in detail, along with an explanation and demonstration of the algorithm. Asymptotic notation, which is presented in Section III, can be used to represent the algorithm's complexity. Section IV describes the experiment and result analysis, while Section V contains the paper's conclusion and recommendations for further research.

II. PROPOSED ALGORITHM

This section presents the complete procedure of the proposed algorithm with three subsections including an algorithm with algorithmic explanations, an illustration of the proposed algorithm, and theoretical evaluation. The first subsection describes two algorithms where Algorithm-1 is a modified bubble sort and Algorithm-2 is modified insertion sort. The second subsection proves the authenticity of the proposed algorithm through a theoretical explanation. On the other hand, the third subsection describes the complete procedure step by step with a proper illustration.

A. Proposed Algorithms

The entire process is structured around two algorithms. The entire operation is shown to revolve around the Algorithm-2, which acts as a caller function. Algorithm-1 is a call function and Algorithm-2 is a caller function. Algorithm -1 is called by Algorithm-2. Our suggested algorithm's working procedure has been broken down into a few phases for clarity.

Algorithm 1 Modified Bubble (N, blockLn, blockSz)

```
1:  for i ← 1 to blockSz do
2:      idx ← (i − 1) ∗ blockLn
3:      for j ← 1 to blockLn do
4:          for k ← (idx + 1) to (idx + blockLn − 1) do
5:              if k >= N then
6:                  break
7:              end if
8:              if A[k] > A[k + 1] then
9:                  swap(A[k], A[k + 1])
10:             end if
11:         end for
12:     end for
13:     Block[i].value ← A[idx + 1]
14:     Block[i].index ← idx + 1
15:     Block[i].last ← min(idx + blockln, N)
16: end for
```

Algorithm 2 Modified Insertion (A)

```
1:  N ← A.length
2:  blockSz ← floor(sqrt(N)) ← numberofblock
3:  blockLn ← ceil(N/blockSz)
4:  ModifiedBubble(N, blockLn, blockSz)
5:  for i ← 1 to N do
6:      crnMin ← inf
7:      for j ← 1 to blockSZ do
8:          if Block[j].value < curMin then
9:              curMin ← Block[j].value
10:             ix ← Block[j].index
11:             id ← j
12:         end if
13:     end for
14:     Output[i] ← curMin
15:     if  ix + 1 > Block[id].last then
16:         Block[id].value ← inf
17:         continue
18:     end if
19:     Block[id].value ← A[ix + 1]
20:     Block[id].index + +
21: end for
```

- Algorithm-1 is used to sort each block individually using the bubble sort algorithm. The total number of block(blockSz) and size of each block(blockLn) are obtained from Algorithm-2. Algorithm-1 line 2 indicates the last index of the previous block. Lines 3 to 11 are used

to sort block i using bubble sort from $idx+1$ to the last index of block i. As idx is the last index of the previous block $(i-1)$, so $idx+1$ is the first index of the $i'th$ block. As well as, the last index of block i is calculated by adding the block length with idx. After being sorted the first index of the $i'th$ block must contain minimum elements of that block with index number $idx+1$. Lines 13 to 15 are used to save the minimum value with its current minimum index and the last index of $i'th$ block to $Block[i]$.

- The structural array $Block$ contains three types of data at each of its indexes.For any index k of Block, $Block[k]$.value contains the minimum value of kth block and $Block[k]$. Index i contains the index of that minimum value. Similarly $Block[k]$, last is the last index of the kth block. It also contains the last index of that block. So, from Algorithm-1 an array $Block$ is built. After that Algorithm-2 is used to sort Block arrays using insertion sort.
- Algorithm-2, Line 4 is used for calling the Algorithm-1 that process on the input array A to make some blocks and sort them individually by using the bubble sort and also save some information for each block in the array named $Block$. Lines 7 to 13 are used to find the current minimum value among all of the blocks and store that value into the $Output$ array. Lines 15 to 20 are used for inserting the next minimum value to the $Block$ from the earlier one comes and also updating the index of that block. If any block is fully used then update the value of that block by infinity max value so that this block is no more going to use. The same procedure is being used for N times. The inserting process is done by the process of modification of insertion sort until to get a full sorted $Output$ array.

B. Theoretical Evaluation

Theoretical evaluation is proof of the existence of the proposed algorithm. Theoretical evaluation argues in favor of the proposed algorithm with a proper theoretical explanation. In this section, two phases are described for Algorithm-1 and Algorithm-2.

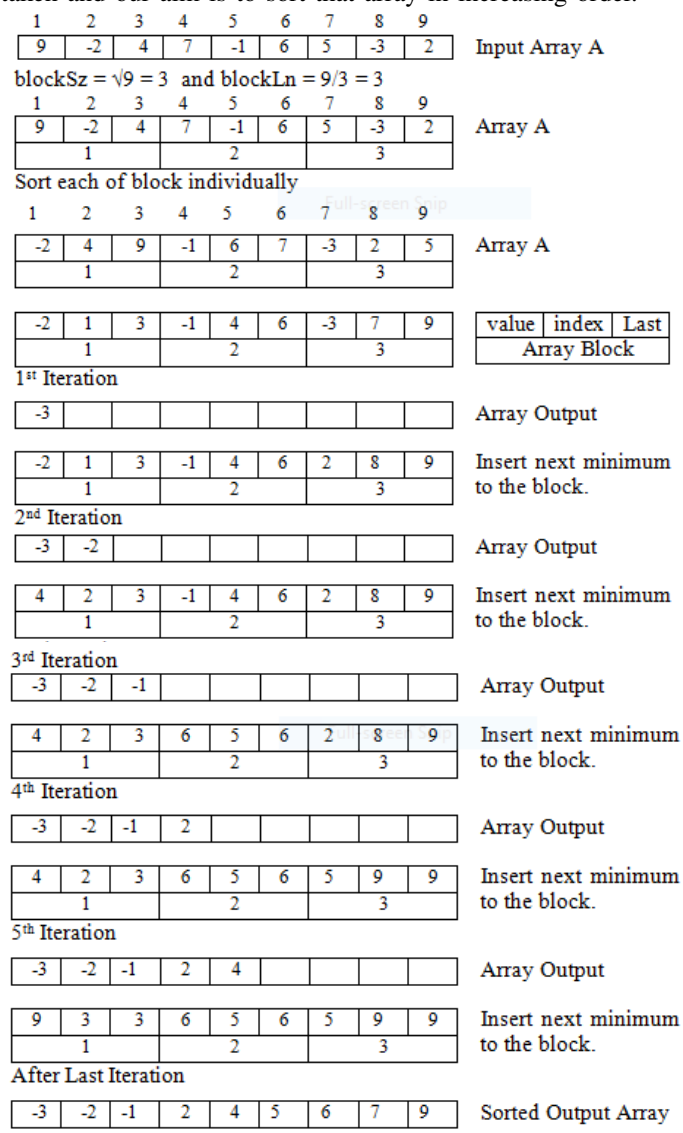
Phase 1: It talks about the modified bubble sort where the input array is divided into $\sqrt{N}$ blocks. Each block consists of approximately $\sqrt{N}$ elements. Then each block has to be sorted using the bubble sort algorithm. After that, a structural array saves some data. So, total $(\sqrt{N})^2 \simeq N$ complexity needed to sort a single block and for overall $\sqrt{N}$ blocks total cost is $N\sqrt{N}$.

Phase 2: It talks about Algorithm-2 where N iterations have occurred. From each iteration, the current minimum value is found, and insert that value to the output array and store the next minimum value using a modified insertion technique to the structural array named $Block$. As there are total N iterations and in each iteration used to find the current minimum from $\sqrt{N}$ block, the overall complexity for this Phase is $N\sqrt{N}$.

The two above Phases of analysis prove that the proposed method is not an unrealistic thought. Moreover, Complexity analysis is presented in Section IV which proves the performance of this algorithm.

C. Illustration

This subsection is designed for illustrating the proposed algorithm with a proper simulation. An input array named A is taken and our aim is to sort that array in increasing order.



Firstly, array A with nine elements is divided into three blocks and then sort each of the three blocks individually. After that, a class or a structural array named $Block$ is taken to save a few data like the current minimum value, index of current minimum value, and last index of that block. After that, it needs to execute nine iterations where each iteration finds the current minimum value from three indexes of the array $Block$. From the example, in the first iterations find the minimum value from the $3rd$ block. Also, update the current value by the next minimum value of that block. Similarly in the second iteration find the minimum value for the $1st$

block and update that block as well for the next iteration. In the same way total of nine iterations are accrued. Finally result from each iteration is inserted into the $Output$ array. As there are a total of nine elements in the input array so, it takes a total of nine iterations.

For a better explanation, five iterations out of nine are showed. To sort the input array in decreasing order the array Block needs to be modified slightly. It should save the maximum value rather than the minimum one and the rest of the procedure is same.

III. COMPLEXITY ANALYSIS

This section presents the overall time complexity with proper explanation. An asymptotic analysis of Algorithm-1 and Algorithm-2 are described to understand the accurate complexity of the proposed algorithm. Table-1 is used for complexity analysis of Algorithm-1 and Table-2 is for Algorithm-2. Each table measures the time of each line of the algorithms. And finally, some mathematical calculations are used for finding overall complexity using the time of each algorithm.

Complexity analysis of Algorithm-1:
As the input array is divided into $\sqrt{N}$ number of partitions and the Algorithm-1 is used to sort each of the partitions individually. Here N is the total number of elements of the input array. As we know, to sort an array of n elements, the time complexity of bubble sort algorithm is $\mathcal{O}(n^2)$. In our proposed algorithm an array of N elements can divided by $\sqrt{N}$ partitions and each partition consists of approximately $\sqrt{N}$ elements. So, when it comes to sort a single partition using bubble sort, it take $\mathcal{O}((\sqrt{N})^2)$ or $\mathcal{O}(N)$ time. And to sort each of $\sqrt{N}$ partitions individually, it takes overall $\mathcal{O}(N\sqrt{N})$ time complexity.

TABLE I: Complexity Analysis of Algorithm-1

Line No	Time
1 - 2	$\sqrt{N}$
3 (Nested loop of line 1)	$\sqrt{N}\sqrt{N} = N$
4 - 10 (Nested loop of line 3)	$\sqrt{N}\sqrt{N}\sqrt{N} = N\sqrt{N}$
13 - 15	$\sqrt{N}$

Table-1 describes the complexity of Algorithm-1 by calculating the time for each line. The time denoted as N where N is the total number of elements. The overall complexity of Algorithm-1 will be the maximum time find from table-1 and that is $N\sqrt{N}$. So, The time complexity of Algorithm-1 is $\mathcal{O}(N\sqrt{N})$

Complexity analysis of Algorithm-2: From Algorithm-1 a structural array named $Block$ is created and save some information. $Block.value$ saves the minimum value , $Block.index$ saves the index of that minimum value and $Block.last$ indicates the last index of a particular block. Basically, Algorithm-2 works on that $Block$ array by doing the same work in N times. In each of N iteration the algorithm-2 find the current minimum value over $\sqrt{N}$ blocks and update the information of a particular block of the array $Block$. It takes $\sqrt{N}$ time to find and update the information of $Block$ because the size of the array $Block$ is equal to the number of blocks($\sqrt{N}$). And as total N iterations are used so total time complexity is $\mathcal{O}(N\sqrt{N})$.

TABLE II: Complexity Analysis of Algorithm- 2

Line No	Time
1 - 3	1
4 (Called Algorithm-1)	N$\sqrt{N}$
5 - 6	N
7 - 12 (Nested loop of line 5)	N$\sqrt{N}$
14 - 20	N

Table-2 describes the complexity of Algorithm-2. From the table-2 the maximum time taken in table-2 is $N\sqrt{N}$
So, The time complexity of Algorithm-2 is $\mathcal{O}(N\sqrt{N})$

It is noticed that the maximum complexity of both Algorithm-1 and Algorithm-2 is $\mathcal{O}(N\sqrt{N})$ individually and they works separately. So, the time complexity of our proposed algorithm is $\mathcal{O}(N\sqrt{N})$

IV. EXPERIMENTAL SETUP AND RESULTS

Algorithms presented in Section II were implemented using C++ programming language on 64bit windows Operating System, Intel Core-i5 processor, 4GB RAM computer.

Firstly, the proposed method have checked on a sorting problem of an online platform. The submitted solution tested for 100000 data and the proposed method successfully passed all the data-sets by taking only 109 ms or 0.109 s. On the other hand any existing sorting algorithm of $\mathcal{O}(N^2)$ did not pass within the given time limit(1s). Because to executed a for loop from 1 to $(100000)^2$ takes almost 100s in a general processor. These are the online data-sets. But the proposed method also checked using custom data-sets also.

Seven different data-sets have been used to experiment with the proposed algorithms. Each of the data-set is generated with a list of a random value and the number of instances is 10000 to 70000 from data-set 1 to data-set 7. Column-1 and Column-2 of Table III show the description of the data-set. Experimental results are also compared with some mentioned sorting algorithm presented in Table III.

Table III presents the real execution time in milliseconds for the proposed method and the mentioned algorithms for the data-sets. The experiment result of the proposed method is compared with some comparison based algorithms insertion, selection, and bubble sort algorithm. The proposed algorithm is the improved and modified version of bubble sort and insertion sort that is mentioned earlier which shows better results

TABLE III: Result Comparison (milliseconds)

Data Set	No.of instance	Selection Sort	Bubble Sort	Insertion Sort	Proposed method
Set-1	10000	0.1870	0.6280	0.1930	0.0020
Set-2	20000	0.7940	2.0280	0.8990	0.0160
Set-3	30000	1.6030	4.4910	1.8280	0.0320
Set-4	40000	2.7280	7.6130	3.1030	0.0430
Set-5	50000	4.1010	11.4900	4.7520	0.0620
Set-6	60000	5.8840	16.4350	6.8010	0.0780
Set-7	70000	7.9220	22.3210	9.1360	0.0950

than the existing bubble sort and insertion sort. From the result of Table III, it can be seen that the proposed algorithm performed better than the existing one and some other sorting algorithms also. Moreover, the comparison result shows that the proposed method outperforms insertion, selection, and bubble sort.

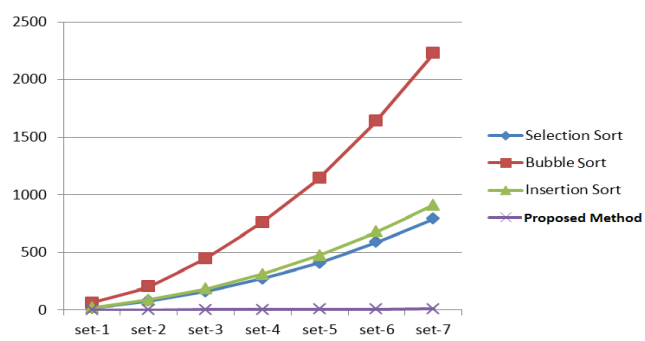


Fig. 1: Comparison of CPU execution time for different sorting algorithm

Figure 1 shows the comparison result for the proposed algorithm with some other algorithms. The execution time measured in milliseconds shown in the vertical axis where different data sets are shown in the horizontal axis. This graph shows that existing algorithms getting worst by increasing the number of data elements. On the contrary, the execution time taken by the proposed method is significantly lower than other sorting algorithms for any cases. Unlike some non-comparison based sorting algorithm such as counting sort, Radix sort the time complexity of the proposed algorithm depends on the number of elements not the type or range of elements. So, the proposed method not only better for the mentioned data-sets only but produces significantly better output for any kind of data-sets.

V. CONCLUSION

Research opportunities are many in computer science. There are several areas where understanding and analyzing algorithms are necessary, including artificial intelligence, database systems, large data analysis, and others. One of the basic algorithms that contribute somewhat to many other academic disciplines is sorting. A hybrid of the bubble sort and insertion sort algorithms is presented in this study. The technique of sorting is based on existing bubble sort and insertion sort, which separates the provided element into numerous blocks, sorts them one at a time using the bubble sort algorithm and then merges them using insertion sort. The theoretical and empirical methodology demonstrates that the complexity of this sorting strategy is $\mathcal{O}(N\sqrt{N})$. Our motivation is not to replace this technique with all other existing sorting algorithms and we don't claim that the proposed methodology outperforms all the sorting techniques. Our claim is that the proposed methodology is much better than bubble sort and insertion sort. Additionally, the result section displays how various scenarios performed. Future efforts will focus on putting the suggested approach to use, particularly in networking and database systems.

Dataset: https://github.com/Tithi-Paul/Dataset-of-Sorting-Algorithm

REFERENCES

[1] Ye, X., Fan, D., Lin, W., Yuan, N., Ienne, P. (2010, April). High performance comparison-based sorting algorithm on many-core GPUs. In 2010 IEEE International Symposium on Parallel Distributed Processing (IPDPS) (pp. 1-10). IEEE.

[2] Roy, H., Shafiuzzaman, M., Samsuddoha, M. (2019, December). SRCS: A New Proposed Counting Sort Algorithm based on Square Root Method. In 2019 22nd International Conference on Computer and Information Technology (ICCIT) (pp. 1-6). IEEE.

[3] Al-Kharabsheh, K. S., AlTurani, I. M., AlTurani, A. M. I., Zanoon, N. I. (2013). Review on sorting algorithms a comparative study. International Journal of Computer Science and Security (IJCSS), 7(3), 120-126.

[4] Kazim, A. (2017). A comparative study of well known sorting algorithms. International Journal of Advanced Research in Computer Science, 8(1).

[5] Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms. MIT press.

[6] Moses, O. O. Performances of Modified Diminishing Increment Sorting In Improving the Performances of Some Sorting Algorithms.

[7] Mundra, J., Pal, M. B. (2015). Minimizing Execution Time of Bubble Sort Algorithm. International Journal of Computer Science and Mobile Computing, 4(9), 173-181.

[8] Moses, O. O. (2009). Improving the performance of bubble sort using a modified diminishing increment sorting. Scientific Research and Essays, 4(8), 740-744.

[9] Jadoon, S., Solehria, S. F., Rehman, S., Jan, H. (2011). Design and analysis of optimized selection sort algorithm. International Journal of Electric Computer Sciences (IJECS-IJENS), 11(01), 16-22.

[10] Kruse R., and Ryba A., Data Structures and Program Design in C++, Prentice Hall, 1999.

[11] Shahzad B. and Afzal M., "Enhanced Shell Sorting Algorithm," Computer Journal of Enformatika, vol. 21, no. 6, pp. 66-70, 2007.

[12] Cormen T., Leiserson C., Rivest R., and Stein C., Introduction to Algorithms, McGraw Hill, 2001.

[13] Aho A., Hopcroft J., and Ullman J., The Design and Analysis of Computer Algorithms, Addison Wesley, 1974.

[14] Thorup M., "Randomized Sorting in O(n log logn) Time and Linear Space Using Addition, Shift, and Bit Wise Boolean Operations," Computer Journal of Algorithms, vol. 42, no. 2,pp. 205-230, 2002.

[15] C. A. R. Hoare, Algorithm 64: Quick sort. Comm. ACM, vol. 4, no. 7 (1961), pp. 321.

[16] Sedge wick, Algorithms in C++, pp.98-100, ISBN 0-201-51059-6, Addison-Wesley, 1992

[17] Hammad, J. (2015). A comparative study between various sorting algorithms. International Journal of Computer Science and Network Security (IJCSNS), 15(3), 11.

Source Code: https://codeforces.com/contest/451/submission/114695225